Evaluación Parcial 1

Ejecución práctica

Instrucciones y pauta de evaluación

Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
DSY1105	Desarrollo de Aplicaciones Móviles	1 semana/15 min	30%

1. Situación Evaluativa 1:

x	Ejecución práctica

2. Instrucciones generales

Descripción
▪ La evaluación parcial 1 consiste en el desarrollo de una aplicación de consola funcional que resuelva un caso de estudio asignado por el/la docente. El proyecto integrará todos los aprendizajes de la primera experiencia, incluyendo la aplicación de variables y condicionales, la creación de funciones, la manipulación de colecciones, el diseño de un sistema de clases con herencia y polimorfismo, y la implementación de una tarea asíncrona con corrutinas junto a funcionalidades avanzadas. Se entregará un caso de negocio para su resolución en una sesión única para medir las competencias de codificación adquiridas por el estudiante a lo largo de la primera unidad. ▪ El tiempo asignado para el desarrollo de esta evaluación en el **taller de alto cómputo** es de **tres bloques pedagógicos**. El tiempo restante (aproximadamente **2 horas**) se destinará a la **retroalimentación** y a la **revisión del trabajo correspondiente a la Experiencia 1**.

- La ejecución de esta actividad se llevará a cabo durante la **semana 5**, momento en el cual se entregará el **caso de negocio**. Los/as estudiantes contarán con toda la sesión para desarrollar su trabajo.
- El trabajo es de **carácter individual** y su **evaluación práctica** se realizará en base al **cumplimiento de los requerimientos establecidos en el caso de negocio**.
- Las instrucciones se proporcionarán el día de la implementación de esta evaluación, considerando los siguientes indicadores de logro:
  - *IL1.1 Compara los tipos de datos en el lenguaje o framework, evaluando la efectividad del uso de operadores aritméticos y condicionales para resolver problemas matemáticos específicos, explicando las diferencias y similitudes en su uso y manejo.*
  - *IL1.2 Desarrolla funciones y colecciones, realizando operaciones utilizando ciclos e iteraciones, mostrando un entendimiento claro de la estructura Front-End, según los requerimientos del cliente.*
  - *IL1.3 Desarrolla un programa orientado a objetos, respetando el patrón arquitectónico y utilizando principios de clases, herencia y polimorfismo, incluyendo el manejo de excepciones y verificando su correcto funcionamiento en un entorno de pruebas.*

Instrucciones específicas de la Evaluación

Consideraciones para la realización de la ejecución práctica:

Se recomienda realizar esta evaluación en laboratorio o sala equipada con computadores, emuladores o dispositivos físicos configurados, acceso a Intellij IDEA, conexión a internet y visibilidad compartida de pantalla.

 Requisitos obligatorios de la evaluación:

La solución debe incluir lo siguiente:

- Aplicación de Lógica y Fundamentos Básicos: Se evaluará una resolución que demuestre el uso correcto de variables y tipos de datos, junto con la aplicación de operadores aritméticos, condicionales y lógicos, para implementar una lógica que procese información y determine un resultado basado en múltiples condiciones.
- Modelado con Programación Orientada a Objetos: Se revisará la correcta implementación de una clase base (open class), al menos dos clases hijas que hereden de ella y la sobrescritura (override) de un método para demostrar polimorfismo.
- Manipulación Funcional de Colecciones: Se verificará el uso de una List para almacenar los objetos y la aplicación de funciones de orden superior (como filter, map o sumOf) para analizar y transformar los datos de la colección.
- Implementación de Tarea Asíncrona: Se evaluará la creación de una suspend fun que utilice delay para simular una operación lenta. Además, se revisará el manejo de los resultados de esta corrutina.

- **Manejo de Estados con Clases Selladas:** Se verificará el uso de una sealed class para modelar los diferentes estados posibles de la operación asíncrona, y su correcta gestión a través de una expresión when.
- **Código Robusto y Manejo de Errores:** Se revisará la implementación de un mecanismo (como try-catch) para controlar datos inválidos (ej. un precio negativo) y evitar que la aplicación falle.
- **Estructura y Calidad del Código:** Se evaluará que el código esté organizado en funciones con responsabilidades claras, sea legible y esté debidamente comentado en las secciones clave.

Forma de entrega:

- El proyecto al finalizar debe ser comprimido y subido a la plataforma antes de finalizar la evaluación.
- El archivo comprimido debe contener todo lo desarrollado en la sesión
- Se subirá el archivo a la plataforma AVA todo el código fuente desarrollado por el estudiante. **En caso de no entregar en AVA no se podrá realizar por ningún otro medio ni después del término de la sesión y se le generará un 1.0 de manera automática**.

Consideraciones para todos los estudiantes:

- Puede utilizar apuntes escritos (no digitales).
- Limitar el uso de internet en el aula para evitar cualquier uso de herramientas de apoyo.
- El/la docente debe monitorear evaluación y trabajo continuo del estudiante desde las herramientas aplicadas en el equipo docente.
- No se admite el uso de IA o recursos de apoyo durante el desarrollo de la evaluación.

3. Pauta de Evaluación

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	80%	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta			Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	Desempeño aceptable 60%	Desempeño no logrado 0%	
IE 1.1.1 Selecciona tipos de datos y variables en Kotlin para almacenar información numérica y textual, considerando su pertinencia según el problema.	Selecciona y declara variables utilizando los tipos de datos más adecuados (Int, Double, String, etc.) según los requerimientos del problema, aplicando correctamente val y var según la mutabilidad necesaria	Selecciona y declara variables, la mayoría de los tipos de datos utilizados son correctos, pero existe algún error significativo (ej. usar Int para un precio).	No declara las variables o son completamente incorrectas.	10%
IE 1.1.2 Aplica operadores aritméticos en Kotlin para realizar cálculos, considerando el tipo de dato involucrado en el problema.	Aplica correctamente todos los cálculos aritméticos necesarios con una lógica clara y eficiente.	Aplica los cálculos, pero el resultado es incorrecto debido a un error lógico.	No implementa los cálculos solicitados.	10%
IE 1.1.3 Implementa estructuras condicionales (if/else, when) en Kotlin	Implementa estructuras condicionales (if/else o when) de forma efectiva para controlar el flujo del programa y	Implementa estructuras condicionales, pero estas son	No utiliza estructuras condicionales.	10%

para controlar el flujo del programa en función de los datos procesados.	manejar diferentes casos de manera robusta.	incompletas o no cubren todos los casos necesarios.		
IE 1.2.1 Estructura el código en funciones modulares y reutilizables que encapsulan lógica específica, facilitando su comprensión y mantenimiento.	Estructura el código de manera organizada en funciones claras, con nombres descriptivos y una única responsabilidad.	Estructura el código en funciones, pero la lógica está mezclada y no sigue el principio de responsabilidad única.	No se utilizan funciones para estructurar el código.	10%
IE 1.2.2 Utiliza colecciones como List o Map para almacenar y gestionar datos de manera eficiente, considerando su estructura y uso dentro del flujo del programa.	Crea y gestiona correctamente una List para almacenar los objetos del caso de estudio.	Crea y gestiona la colección, la cual es funcional, pero su manejo es ineficiente o presenta errores menores.	No utiliza una colección para gestionar los datos.	10%
IE 1.2.3 Aplica funciones de orden superior (filter, map, sumOf, etc.) para consultar y transformar la colección de datos de forma concisa.	Aplica funciones de orden superior (filter, map, sumOf, etc.) de forma idiomática y eficiente para analizar la colección.	Aplica funciones de orden superior, pero recurre a ciclos for para resolver problemas que serían más simples con el enfoque funcional.	No implementa las operaciones funcionales.	15%
IE 1.2.4 Utiliza estructuras de iteración sobre la colección para mostrar los resultados de forma clara en la consola, simulando la capa de presentación o Front-End.	Utiliza estructuras de iteración para recorrer la colección y mostrar los resultados en consola de forma clara, ordenada y fácil de leer.	La visualización de datos es funcional, pero incompleta o desordenada.	No muestra los resultados de las operaciones.	5%
IE 1.3.1 Implementa una jerarquía de clases basada en herencia, aplicando principios de reutilización y extensión del comportamiento.	Implementa correctamente la herencia con open class y clases hijas, pasando los parámetros al constructor de la superclase.	Implementa la herencia, pero con errores conceptuales graves (ej. la clase base no es open).	No utiliza clases ni herencia.	10%
IE 1.3.2 Aplica polimorfismo sobrescribiendo métodos en una clase derivada, demostrando cómo se modifica el comportamiento heredado.	Aplica polimorfismo mediante la sobrescrita (override) de métodos en clases derivadas, demostrando cómo se modifica el comportamiento heredado a través de instancias polimórficas	Aplica polimorfismo mediante la sobre escritura, pero con errores de sintaxis o sin marcar el método base como open.	No implementa la sobrescritura de métodos.	5%
IE 1.3.3 Implementa una tarea asíncrona simple utilizando corutinas (suspend fun, delay), integrándola de forma coherente en el flujo del programa.	Implementa una función suspend con delay, ejecutada mediante runBlocking, e integrada adecuadamente en el flujo lógico del programa para demostrar comportamiento asíncrono	Implementa la función suspend fun, pero su ejecución desde main es incorrecta o no funciona como se espera.	No implementa la tarea asíncrona	10%

IE 1.3.4 Implementa manejo básico de excepciones utilizando bloques try-catch, asegurando la continuidad del programa ante errores controlados.	Implementa un bloque try-catch para gestionar posibles excepciones de forma robusta, previniendo que la aplicación falle.	Implementa el try-catch pero no captura la excepción correctamente o no maneja el error de forma adecuada.	No implementa manejo de errores.	5%
			Total	**100%**